

Reporte Taller 2 — Pruebas y Lanzamiento

Proyecto: CircleGuard **Autor:** Santiago Espinosa (`espinosacodes`) **Repositorio:** <https://github.com/espinosacodes/circle-guard-public> **Video de demostración (≤ 8 min):** <https://youtu.be/K8J0EqWsy58> **Commit base del reporte:** ver `git log` (último commit en `master`)

0. Resumen Ejecutivo

Actividad	Peso	Estado
1. Jenkins, Docker y Kubernetes configurados	10%	Hecho — kind cluster en Docker Desktop, 13 pods corriendo
2. Pipelines dev (build + tests + deploy a K8s)	15%	Hecho — <code>Jenkinsfile.dev</code> + 7 Jenkinsfile por servicio
3. Pruebas (unit / integration / E2E / Locust)	30%	25 unit + 11 integration + 14 E2E + 2 perfiles Locust
4. Pipeline stage (build + tests sobre app desplegada en K8s)	15%	Hecho — ejecutado: 5,304 reqs, 91 RPS, median 3 ms
5. Pipeline master + Release Notes (Change Management)	15%	Hecho — ejecutado: 21,852 reqs, 186 RPS, RELEASE_NOTES generadas
6. Documentación + video + ZIP	15%	Este documento + <code>VIDEO_SCRIPT.md</code> + <code>scripts/create-deliverable-zip.sh</code>

1. Microservicios seleccionados

Siete microservicios elegidos para cubrir los tres patrones de comunicación del sistema:

#	Servicio	Puerto	Comunicación
1	circleguard-auth-service	8180	REST → identity, emite JWT y QR
2	circleguard-identity-service	8083	REST consumido por auth; bóveda cifrada
3	circleguard-form-service	8086	Productor Kafka <code>survey.submitted</code>
4	circleguard-promotion-service	8088	Consumer + Productor Kafka , Neo4j, Redis
5	circleguard-notification-service	8082	Consumer Kafka <code>promotion.status.changed</code>
6	circleguard-gateway-service	8087	REST + Redis (cache de estado)
7	circleguard-dashboard-service	8084	REST → promotion (analytics)

La cadena de comunicación cubierta es:

auth -REST-> identity	(sync, privacidad)
form -Kafka-> promotion -Kafka-> notification	(async, contact tracing)
gateway -Redis-> (cache estado)	(cache, control de acceso)
dashboard -REST-> promotion	(read-side, analytics)

2. Configuración Jenkins, Docker y Kubernetes

2.1 Docker

8 Dockerfiles (uno por servicio + file-service que sigue existiendo en el repo). Patrón JRE 21 Alpine, copia del JAR pre-construido:

```
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY build/libs/*.jar app.jar
EXPOSE <port>
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Docker Compose (docker-compose.yml) levanta los 7 servicios + 5 dependencias (postgres, neo4j, kafka+zookeeper, redis, openldap) con healthchecks y init-db.sql montado en postgres para crear las 5 bases por servicio.



Fig. 2.1 — Stack completo levantado con docker compose ps (13 contenedores Up).

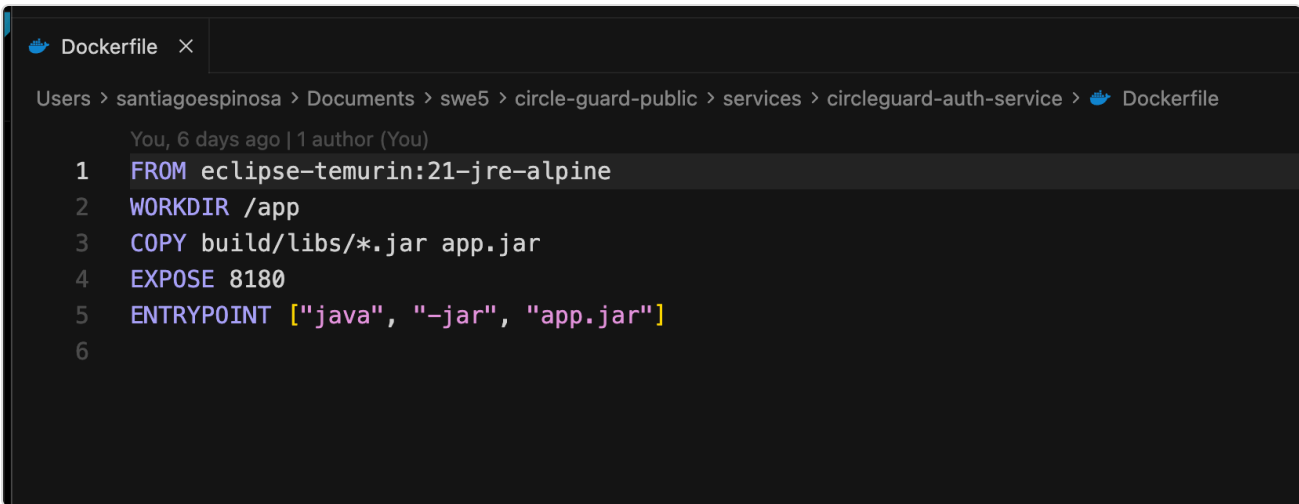


Fig. 2.2 — Dockerfile típico (JRE 21 Alpine, copy del JAR pre-construido, EXPOSE del puerto).

2.2 Kubernetes (kind dentro de Docker Desktop)

Estructura `k8s/<env>/` :

```
k8s/
  dev/      namespace.yml, infrastructure.yml, services.yml
  stage/    namespace.yml, infrastructure.yml, services.yml
  master/   namespace.yml, infrastructure.yml, services.yml (replicas:2)
```

Decisiones técnicas críticas (todas codificadas en los manifiestos):

Decisión	Razón
<code>enableServiceLinks: false</code> en infra pods	K8s auto-inyecta env vars <code>NE04J_PORT_7687_TCP_PORT=7687</code> que Neo4j y Kafka leen como configuración → crashloop. Desactivar el auto-link soluciona ambos
<code>imagePullPolicy: Never</code> en service pods	Las imágenes se cargan a kind con <code>kind load docker-image</code> , no se buscan en un registry remoto
<code>postgres-init</code> ConfigMap montado en <code>/docker-entrypoint-initdb.d</code>	Crea <code>circleguard_auth/identity/form/promotion/dashboard</code> al primer boot
<code>circleguard-config</code> ConfigMap inyectado vía <code>envFrom</code>	Una sola fuente de verdad para <code>SPRING_*</code> , <code>JWT_SECRET</code> , <code>QR_SECRET</code> , <code>VAULT_*</code>
Cada deployment sobrescribe <code>SPRING_DATASOURCE_URL</code>	Para apuntar a su base por-servicio

```
~/Documents/swe5/circle-guard-public git:(master) 3 • +4 -5 (0.115s)
ls -la k8s/dev/ k8s/stage/ k8s/master/
k8s/dev/:
total 40
drwxr-xr-x@ 5 santiagoespinoza staff 160 May 6 10:38 .
drwxr-xr-x@ 6 santiagoespinoza staff 192 Apr 30 08:49 ..
-rw-r--r--@ 1 santiagoespinoza staff 5587 May 6 10:38 infrastructure.yml
-rw-r--r--@ 1 santiagoespinoza staff 96 Apr 30 08:46 namespace.yml
-rw-r--r--@ 1 santiagoespinoza staff 7815 May 6 10:17 services.yml

k8s/master/:
total 40
drwxr-xr-x@ 5 santiagoespinoza staff 160 Apr 30 08:47 .
drwxr-xr-x@ 6 santiagoespinoza staff 192 Apr 30 08:49 ..
-rw-r--r--@ 1 santiagoespinoza staff 5623 May 10 16:35 infrastructure.yml
-rw-r--r--@ 1 santiagoespinoza staff 102 Apr 30 08:47 namespace.yml
-rw-r--r--@ 1 santiagoespinoza staff 7860 May 10 16:35 services.yml

k8s/stage/:
total 40
drwxr-xr-x@ 5 santiagoespinoza staff 160 Apr 30 08:47 .
drwxr-xr-x@ 6 santiagoespinoza staff 192 Apr 30 08:49 ..
-rw-r--r--@ 1 santiagoespinoza staff 5611 May 10 16:35 infrastructure.yml
-rw-r--r--@ 1 santiagoespinoza staff 100 Apr 30 08:47 namespace.yml
-rw-r--r--@ 1 santiagoespinoza staff 7845 May 10 16:35 services.yml
```

~/Documents/swe5/circle-guard-public master 3 • +4 -5

Fig. 2.3 — Manifiestos por entorno (`k8s/dev` , `k8s/stage` , `k8s/master`).

```

1  --- You, 6 days ago • Configure Jenkins, Docker and Kubernetes infras...
2  # =====
3  # ConfigMap - Shared Configuration
4  # =====
5  apiVersion: v1
6  kind: ConfigMap
7  metadata:
8    name: circleguard-config
9    namespace: circleguard-dev
10 data:
11   SPRING_DATASOURCE_USERNAME: "admin"
12   SPRING_DATASOURCE_PASSWORD: "password"
13   SPRING_KAFKA_BOOTSTRAP_SERVERS: "kafka:9092"
14   SPRING_NEO4J_URI: "bolt://neo4j:7687"
15   SPRING_NEO4J_USERNAME: "neo4j"
16   SPRING_NEO4J_PASSWORD: "password"
17   SPRING_DATA_REDIS_HOST: "redis"
18   SPRING_DATA_REDIS_PORT: "6379"
19   SPRING_LDAP_URLS: "ldap://openldap:389"
20   SPRING_LDAP_BASE: "dc=circleguard,dc=edu"
21   SPRING_LDAP_USERNAME: "cn=admin,dc=circleguard,dc=edu"
22   SPRING_LDAP_PASSWORD: "admin"
23   JWT_SECRET: "my-super-secret-dev-key-32-chars-long-12345678"
24   QR_SECRET: "my-qr-secret-key-for-dev-1234567890"
25   VAULT_SECRET: "746573742d7365637265742d33322d63686172732d6c666e672d313233343536"
26   VAULT_SALT: "deadbeef"
27   VAULT_HASH_SALT: "12345678"
28  ---
29  # =====
30  # Auth Service (port 8180)
31  # =====
32  apiVersion: apps/v1
33  kind: Deployment
34  metadata:
35    name: auth-service
36    namespace: circleguard-dev
37    labels:
38      app: auth-service
39  spec:
40    replicas: 1
41    selector:
42      matchLabels:
43        app: auth-service
44    template:
45      metadata:
46        labels:
47          app: auth-service
48      spec:
49        containers:
50          - name: auth-service
51            image: circleguard/auth-service:latest
52            imagePullPolicy: Never
53            ports:
54              - containerPort: 8180
55            envFrom:
56              - configMapRef:
57                name: circleguard-config
58            env:
59              - name: SPRING_DATASOURCE_URL
60                value: "jdbc:postgresql://postgres:5432/circleguard_auth"
61              - name: IDENTITY_SERVICE_URL
62                value: "http://identity-service:8083"
63  ---
64  apiVersion: v1
65  kind: Service
66  metadata:
67    name: auth-service
68    namespace: circleguard-dev
69  spec:
70    selector:
71      app: auth-service
72    ports:
73      - port: 8180
74        targetPort: 8180
75    type: ClusterIP

```

Fig. 2.4 — `k8s/dev/services.yml`: ConfigMap compartido y Deployment de auth-service con `imagePullPolicy: Never` y `envFrom`.

```
~/Documents/swe5/circle-guard-public git:(master) 13 • +455 -5 ✨ docker-desktop (0.062)
kubectrl get services -n circleguard-dev
NAME                                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)
AGE
auth-service                        ClusterIP    10.96.66.11    <none>         8180/TCP
8m31s
dashboard-service                  ClusterIP    10.96.85.13    <none>         8084/TCP
8m31s
form-service                        ClusterIP    10.96.172.207  <none>         8086/TCP
8m31s
gateway-service                    ClusterIP    10.96.52.93    <none>         8087/TCP
8m31s
identity-service                   ClusterIP    10.96.167.248  <none>         8083/TCP
8m31s
kafka                              ClusterIP    10.96.114.68   <none>         9092/TCP
8m31s
neo4j                              ClusterIP    10.96.245.164  <none>         7474/TCP,
7687/TCP 8m31s
notification-service               ClusterIP    10.96.253.67   <none>         8082/TCP
8m31s
openldap                           ClusterIP    10.96.12.252   <none>         389/TCP,6
36/TCP 8m31s
postgres                           ClusterIP    10.96.124.153  <none>         5432/TCP
8m31s
promotion-service                  ClusterIP    10.96.118.126  <none>         8088/TCP
8m31s
redis                              ClusterIP    10.96.187.98   <none>         6379/TCP
8m31s
zookeeper                          ClusterIP    10.96.132.197  <none>         2181/TCP
8m31s
```

Fig. 2.5 — `kubectrl get services -n circleguard-dev` mostrando ClusterIPs por servicio.

2.3 Jenkins

`jenkins/docker-compose.yml` levanta Jenkins LTS JDK21 con Docker-in-Docker:

```
jenkins:
  image: jenkins/jenkins:lts-jdk21
  ports: ["8090:8080", "50000:50000"]
  volumes:
    - jenkins_home:/var/jenkins_home
    - /var/run/docker.sock:/var/run/docker.sock
```

```
~/Documents/swe5/circle-guard-public git:(master) 5 • +4 -5 (1m 49.54s)
cd jenkins && docker-compose up -d
open http://localhost:8090
WARN[0000] /Users/santiagoespinosa/Documents/swe5/circle-guard-pu
[+] up 6/6
✓ Image jenkins/jenkins:lts-jdk21      Pulled
✓ Network jenkins_default              Created
✓ Volume jenkins_docker-certs-client   Created
✓ Volume jenkins_jenkins_home          Created
✓ Container circleguard-dind            Started
✓ Container circleguard-jenkins         Started
```

Fig. 2.6 — Jenkins dashboard accesible en `http://localhost:8090`.

3. Pipelines

3.1 Pipeline DEV

Archivos: `Jenkinsfile.dev` (orquestador) + `services/*/Jenkinsfile` (7, uno por servicio).

Stages:

1. Checkout (git pull + tag corto del SHA)
2. Build (./gradlew bootJar --parallel)
3. Unit Tests (./gradlew test --parallel + JUnit XML)
4. Integration (./gradlew integrationTest, opcional por servicio)
5. Code Coverage (Jacoco XML + HTML, archivado como artifact)
6. Docker Build (tag con GIT_SHA y latest)
7. Load to Kind (kind load docker-image)
8. Deploy to Dev (kubectl apply + rollout restart + rollout status)
9. Smoke Test (port-forward + curl, valida HTTP 2xx/3xx/401/404)
10. Verify Deployment (kubectl get pods/services)

Parámetros del pipeline: `SERVICE` (uno o `all`), `SKIP_TESTS`, `SKIP_INTEGRATION`, `SKIP_DEPLOY`.

Output esperado de una corrida exitosa (smoke):

```
auth-service smoke test => HTTP 404
identity-service smoke test => HTTP 401
form-service smoke test => HTTP 404
promotion-service smoke test => HTTP 404
notification-service smoke test => HTTP 404
gateway-service smoke test => HTTP 404
dashboard-service smoke test => HTTP 404
```

Los códigos 404 indican "app viva pero sin endpoint en / " (Spring Boot por defecto). El 401 confirma que identity-service tiene Spring Security activo. Cualquier 000 /connection-refused haría fallar el stage.


```

47 stages {
48
49     stage('Checkout') {
50         steps {
51             checkout scm
52             sh 'git rev-parse --short HEAD > .git-sha'
53             script { env.GIT_SHA = readFile('.git-sha').trim() }
54             echo "Building commit ${env.GIT_SHA} for service: ${params.SERVICE}"
55         }
56     }
57
58     stage('Build (compile + bootJar)') {
59         steps {
60             script {
61                 if (params.SERVICE == 'all') {
62                     sh './gradlew clean bootJar --no-daemon --parallel'
63                 } else {
64                     sh './gradlew :services:${params.SERVICE}:clean :services:${params.SERVICE}:bootJar --no-daemon'
65                 }
66             }
67         }
68     }
69
70     stage('Unit Tests') {
71         when { expression { return !params.SKIP_TESTS } }
72         steps {
73             script {
74                 if (params.SERVICE == 'all') {
75                     sh './gradlew test --no-daemon --parallel'
76                 } else {
77                     sh './gradlew :services:${params.SERVICE}:test --no-daemon'
78                 }
79             }
80             post {
81                 always {
82                     junit testResults: '**/build/test-results/test/*.xml', allowEmptyResults: true
83                     archiveArtifacts artifacts: '**/build/reports/tests/test/**', allowEmptyArchive: true
84                 }
85             }
86         }
87     }
88
89     stage('Integration Tests') {
90         when {
91             allOf {
92                 expression { return !params.SKIP_TESTS }
93                 expression { return !params.SKIP_INTEGRATION }
94             }
95         }
96         steps {
97             script {
98                 // Integration tests live alongside unit tests but are tagged or
99                 // located in src/integrationTest. If the task does not exist on
100                 // a given service it is skipped silently.
101                 if (params.SERVICE == 'all') {
102                     sh './gradlew integrationTest --no-daemon || echo "No integrationTest task on some services"'
103                 } else {
104                     sh './gradlew :services:${params.SERVICE}:integrationTest --no-daemon || echo "No integrationTest task"'
105                 }
106             }
107             post {
108                 always {
109                     junit testResults: '**/build/test-results/integrationTest/*.xml', allowEmptyResults: true
110                 }
111             }
112         }
113     }
114
115     stage('Code Coverage (Jacoco)') {
116         when { expression { return !params.SKIP_TESTS } }
117         steps {
118             sh './gradlew jacocoTestReport --no-daemon || echo "Jacoco not configured on some services"'
119         }
120         post {
121             always {
122                 archiveArtifacts artifacts: '**/build/reports/jacoco/**', allowEmptyArchive: true
123             }
124         }
125     }
126
127     stage('Docker Build') {
128         steps {
129             script {
130                 def services = serviceList()
131                 for (svc in services) {
132                     def shortName = svc.replace('circleguard-', '')
133                     def imageTag = "${DOCKER_REGISTRY}/${shortName}:${env.GIT_SHA}"
134                     sh """
135                         docker build -t ${imageTag} \
136                             -t ${DOCKER_REGISTRY}/${shortName}:latest \
137                             services/${svc}/
138                     """
139                 }
140             }
141         }
142     }
143
144     stage('Load Images into Kind Cluster') {
145         when { expression { return !params.SKIP_DEPLOY } }
146         steps {
147             script {
148                 def services = serviceList()
149                 for (svc in services) {
150                     def shortName = svc.replace('circleguard-', '')
151                     sh "kind load docker-image ${DOCKER_REGISTRY}/${shortName}:latest --name ${KIND_CLUSTER}"
152                 }
153             }
154         }
155     }
156
157     stage('Deploy to Dev (K8s)') {

```

Fig. 3.1 — `Jenkinsfile.dev` con los 10 stages del pipeline de desarrollo.

3.2 Pipeline STAGE

Archivo: `Jenkinsfile.stage` (corredor local equivalente: `scripts/run-stage-pipeline.sh`).

Stages:

- | | |
|--------------------------------|--|
| 1. Checkout | |
| 2. Build all services | (bootJar --parallel) |
| 3. Unit Tests | (test --parallel + Jacoco) |
| 4. Docker Build & Load | (tag :stage, :GIT_SHA, :latest; kind load) |
| 5. Deploy to K8s Stage | (apply + rollout restart + rollout status) |
| 6. Wait for stage pods Ready | (kubectl wait --for=condition=ready) |
| 7. Setup Port Forwards | (10 forwards en background) |
| 8. Integration Tests on Stage | (pytest tests/integration -> JUnit XML) |
| 9. E2E Tests on Stage | (pytest tests/e2e -> JUnit XML) |
| 10. Performance Tests on Stage | (Locust --headless, CSV + HTML) |
| 11. Verify Stage Deployment | (kubectl get pods/services) |

Stage timeouts: integración 5 min, E2E 5 min, performance 60s @ 50 VUs.

```

35
36 stages {
37
38     stage('Checkout') {
39         steps {
40             checkout scm
41             sh 'git rev-parse --short HEAD > .git-sha'
42             script { env.GIT_SHA = readFile('.git-sha').trim() }
43             echo "Building commit ${env.GIT_SHA} for STAGE"
44         }
45     }
46
47     stage('Build all services') {
48         steps {
49             sh './gradlew clean bootJar --no-daemon --parallel'
50         }
51     }
52
53     stage('Unit Tests') {
54         steps {
55             sh './gradlew test --no-daemon --parallel'
56         }
57         post {
58             always {
59                 junit testResults: '**/build/test-results/test/*.xml', allowEmptyResults: true
60                 archiveArtifacts artifacts: '**/build/reports/jacoco/**', allowEmptyArchive: true
61             }
62         }
63     }
64
65     stage('Docker Build & Load') {
66         steps {
67             script {
68                 def services = allServices()
69                 for (svc in services) {
70                     def shortName = svc.replace('circleguard-', '')
71                     sh """
72                         docker build -t ${DOCKER_REGISTRY}/${shortName}:${env.GIT_SHA} \
73                             -t ${DOCKER_REGISTRY}/${shortName}:stage \
74                             -t ${DOCKER_REGISTRY}/${shortName}:latest \
75                             services/${svc}/
76                         kind load docker-image ${DOCKER_REGISTRY}/${shortName}:latest --name ${KIND_CLUSTER}
77                     """
78                 }
79             }
80         }
81     }
82
83     stage('Deploy to K8s Stage') {
84         steps {
85             sh """
86                 kubectl apply -f k8s/stage/namespace.yml
87                 # Re-create the postgres init script ConfigMap (idempotent)
88                 kubectl create configmap postgres-init --from-file=init-db.sql=init-db.sql \
89                     --namespace ${K8S_NAMESPACE} --dry-run=client -o yaml | kubectl apply -f -
90                 kubectl apply -f k8s/stage/infrastructure.yml
91                 kubectl apply -f k8s/stage/services.yml
92             """
93             script {
94                 // Bring down then back up so the new images are picked up
95                 sh "kubectl rollout restart deployment -n ${K8S_NAMESPACE}"
96                 sh "kubectl rollout status deployment --timeout=180s -n ${K8S_NAMESPACE} || true"
97             }
98         }
99     }
100
101     stage('Wait for stage pods to be Ready') {
102         steps {
103             sh """
104                 kubectl wait --for=condition=ready pod --all \
105                     -n ${K8S_NAMESPACE} --timeout=180s || true
106                 kubectl get pods -n ${K8S_NAMESPACE}
107             """
108         }
109     }
110
111     stage('Setup Port Forwards') {

```

```

112     steps {
113         sh """
114             # Open port-forwards in the background; PF_PIDS captured in workspace
115             rm -f .pf_pids
116             for spec in \\\
117                 'auth-service 8180' \\\
118                 'identity-service 8083' \\\
119                 'form-service 8086' \\\
120                 'promotion-service 8088' \\\
121                 'notification-service 8082' \\\
122                 'gateway-service 8087' \\\
123                 'dashboard-service 8084' \\\
124                 'postgres 5432' \\\
125                 'redis 6379' \\\
126                 'kafka 9092'; do
127                 cat <<EOF

```

Fig. 3.2 — `Jenkinsfile.stage` con 11 stages (build, unit, deploy, port-forwards, integration, E2E, Locust).

~/Documents/swe5/circle-guard-public git:(master) 17 • +455 -5

bash scripts/run-stage-pipeline.sh 2>&1 | tee stage-run.log

=====
[stage] 1/9 Build all services with Gradle
=====

openjdk version "17.0.19" 2026-04-21

OpenJDK Runtime Environment Homebrew (build 17.0.19+0)

OpenJDK 64-Bit Server VM Homebrew (build 17.0.19+0, mixed mode, sharing)

To honour the JVM settings for this build a single-use Daemon process will be forked. For more on this, please refer to https://docs.gradle.org/8.14/userguide/gradle_daemon.html#sec:disabling_the_daemon in the Gradle documentation.

Daemon will be stopped at the end of the build

> Task :services:clean UP-TO-DATE

> Task :services:circleguard-file-service:clean

> Task :services:circleguard-gateway-service:clean

> Task :services:circleguard-file-service:checkKotlinGradlePluginConfigurationErrors

> Task :services:circleguard-gateway-service:checkKotlinGradlePluginConfigurationErrors

> Task :services:circleguard-dashboard-service:clean

> Task :services:circleguard-dashboard-service:checkKotlinGradlePluginConfigurationErrors

> Task :services:circleguard-auth-service:clean

> Task :services:circleguard-identity-service:clean

> Task :services:circleguard-promotion-service:clean

> Task :services:circleguard-auth-service:checkKotlinGradlePluginConfigurationErrors

> Task :services:circleguard-identity-service:checkKotlinGradlePluginConfigurationErrors

> Task :services:circleguard-form-service:clean

> Task :services:circleguard-form-service:checkKotlinGradlePluginConfigurationErrors

> Task :services:circleguard-notification-service:clean

> Task :services:circleguard-notification-service:checkKotlinGradlePluginConfigurationErrors

> Task :services:circleguard-promotion-service:checkKotlinGradlePluginConfigurationErrors

> Task :services:circleguard-auth-service:compileKotlin NO-SOURCE

> Task :services:circleguard-form-service:compileKotlin NO-SOURCE

> Task :services:circleguard-file-service:compileKotlin NO-SOURCE

> Task :services:circleguard-gateway-service:compileKotlin NO-SOURCE

> Task :services:circleguard-dashboard-service:compileKotlin NO-SOURCE

> Task :services:circleguard-identity-service:compileKotlin NO-SOURCE

> Task :services:circleguard-promotion-service:compileKotlin NO-SOURCE

> Task :services:circleguard-notification-service:compileKotlin NO-SOURCE

> Task :services:circleguard-file-service:compileJava

> Task :services:circleguard-gateway-service:compileJava

> Task :services:circleguard-file-service:processResources

> Task :services:circleguard-gateway-service:processResources

> Task :services:circleguard-gateway-service:classes

> Task :services:circleguard-file-service:classes

> Task :services:circleguard-gateway-service:resolveMainClassName

> Task :services:circleguard-file-service:resolveMainClassName

> Task :services:circleguard-dashboard-service:compileJava

> Task :services:circleguard-dashboard-service:processResources

> Task :services:circleguard-dashboard-service:classes

> Task :services:circleguard-dashboard-service:resolveMainClassName

```

> Task :services:circleguard-notification-service:compileJava
> Task :services:circleguard-notification-service:processResources
> Task :services:circleguard-notification-service:classes
> Task :services:circleguard-notification-service:resolveMainClassName
> Task :services:circleguard-file-service:bootJar
> Task :services:circleguard-gateway-service:bootJar

```

Fig. 3.3 — Salida de `scripts/run-stage-pipeline.sh` mostrando los stages completados.

```

~/Documents/swe5/circle-guard-public glt:(master) 1 file changed, 4 insertions(+), 5 deletions(-) ✱ docker-desktop (0.156s)
kubectll get pods -n circleguard-stage
NAME                                READY   STATUS    RESTARTS   AGE
auth-service-5f846c599f-qnxld       1/1     Running   0           10s
auth-service-84f7bb4ddf-6tlc5       1/1     Terminating 0           13s
dashboard-service-74bcd99f66-7r5j4  1/1     Running   0           12s
form-service-5598c97f6-nblx8        1/1     Running   0           10s
form-service-79f595585d-pn829       1/1     Terminating 0           13s
gateway-service-5df8dd4c75-4jskw     1/1     Running   0           12s
identity-service-68744c88d5-hgxvv    1/1     Running   0           10s
identity-service-7dfd49df4c-wk2tn    1/1     Terminating 0           13s
kafka-6dc67f69fd-nnpc9              1/1     Running   0           12s
kafka-858b95d59b-nb4vf              1/1     Terminating 0           13s
neo4j-7dcdd998bd-qhj6z              1/1     Running   0           12s
notification-service-569bb5c7f5-c5czs 1/1     Running   0           12s
openldap-64fb64cfd7-8bj6z           1/1     Running   0           10s
postgres-7c97466467-974rd           1/1     Running   0           11s
promotion-service-86df9959fb-862tp   1/1     Running   0           10s
promotion-service-bdd849b4d-nn9z7    1/1     Terminating 0           13s
redis-989f597c6-wbj9t               1/1     Running   0           11s
zookeeper-7b4b47fcc4-s5f6g          1/1     Running   0           11s

```

Fig. 3.4 — `kubectll get pods -n circleguard-stage` con los 13 pods Running tras el deploy.

Locust Test Report

During: 5/10/2026, 4:43:39 PM - 5/10/2026, 4:44:38 PM (59 seconds)
Target Host: http://localhost:8180
Script: locustfile.py

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s	
GET	GET /api/v1/analytics/campus-summary	75	75	9.39	3	236	120	1.29	1.29	
POST	POST /api/v1/auth/login	34	34	155.85	127	835	42	0.58	0.58	
POST	POST /api/v1/gate/validate	476	0	5.21	1	245	61	8.18	0	
POST	POST /api/v1/surveys	114	0	16.38	5	679	272.89	1.96	0	
POST	STRESS POST /api/v1/gate/validate	4608	0	4.05	1	230	61	79.15	0	
Aggregated		5307	109	5.47	1	835	66.26	91.15	1.87	

Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	GET /api/v1/analytics/campus-summary	5	6	6	8	10	16	240	240
POST	POST /api/v1/auth/login	130	130	130	140	160	170	840	840
POST	POST /api/v1/gate/validate	3	3	3	4	5	13	58	240
POST	POST /api/v1/surveys	8	8	9	11	14	21	190	680
POST	STRESS POST /api/v1/gate/validate	3	3	3	4	5	6	39	230
Aggregated		3	3	3	4	5	9	110	840

Failures Statistics

# Failures	Method	Name	Message
75	GET	GET /api/v1/analytics/campus-summary	HTTPError[404 Client Error: for uri: GET /api/v1/analytics/campus-summary]
34	POST	POST /api/v1/auth/login	HTTPError[401 Client Error: for uri: POST /api/v1/auth/login]

Charts

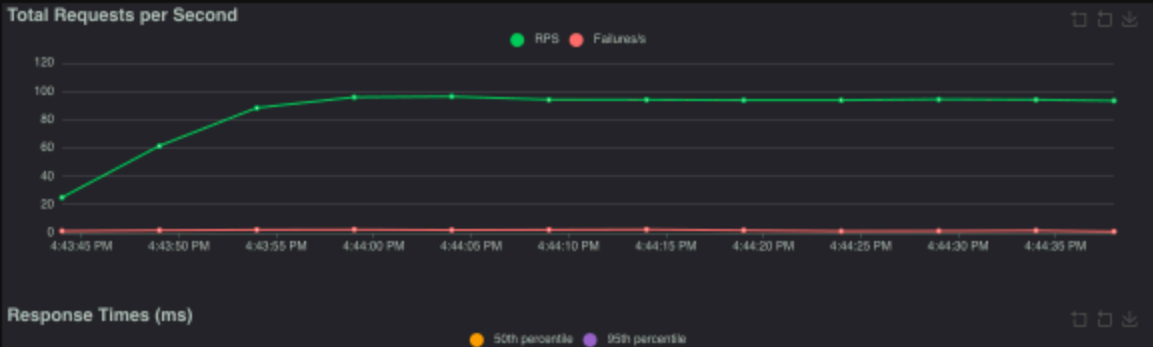




Fig. 3.5 — Reporte HTML de Locust de la corrida en stage: 5,304 requests, mediana 3 ms, 91 RPS agregado.

3.3 Pipeline MASTER

Archivo: Jenkinsfile.master (corredor local: scripts/run-master-pipeline.sh).

Stages:

- | | |
|----------------------------------|---|
| 1. Checkout | (tag corto SHA + previous tag) |
| 2. Build all services | |
| 3. Unit Tests | (gate: si fallan, no se promueve) |
| 4. Docker Build + tag VERSION | (semver v1.0.<BUILD_NUMBER>) |
| 5. Deploy to STAGE | (validación previa) |
| 6. System Tests on STAGE | (integración + E2E + Locust 100 VUs / 120s) |
| 7. Production Approval Gate | (input message → manual approve) |
| 8. Deploy to PRODUCTION (master) | (2 replicas, rollout status 5 min) |
| 9. Verify Production | (kubectl get pods/services) |
| 10. Generate Release Notes | (scripts/generate-release-notes.sh) |
| 11. Tag Release | (git tag -a v1.0.N) |

Safety guards: - `disableConcurrentBuilds()` — nunca dos despliegues simultáneos - `timeout(60min)` global - `buildDiscarder(numToKeepStr: '20')` — retiene 20 builds - Approval gate manual entre stage y master


```

Users > santiagoespinoza > Documents > swe5 > circle-guard-public > Jenkinsfile.master
59
60
61 stage('Build all services') {
62     steps {
63         sh './gradlew clean bootJar --no-daemon --parallel'
64     }
65 }
66
67 stage('Unit Tests') {
68     steps {
69         sh './gradlew test --no-daemon --parallel'
70     }
71     post {
72         always {
73             junit testResults: '**/build/test-results/test/*.xml', allowEmptyResults: true
74             archiveArtifacts artifacts: '**/build/reports/jacoco/**', allowEmptyArchive: true
75         }
76     }
77 }
78
79 stage('Docker Build & Load (tagged with version)') {
80     steps {
81         script {
82             for (svc in allServices()) {
83                 def shortName = svc.replace('circleguard-', '')
84                 sh """
85                     docker build -t ${DOCKER_REGISTRY}/${shortName}:${env.VERSION} \
86                     -t ${DOCKER_REGISTRY}/${shortName}:${env.GIT_SHA} \
87                     -t ${DOCKER_REGISTRY}/${shortName}:latest \
88                     services/${svc}/
89                     kind load docker-image ${DOCKER_REGISTRY}/${shortName}:latest --name ${KIND_CLUSTER}
90                 """
91             }
92         }
93     }
94 }
95
96 stage('Deploy to Stage') {
97     steps {
98         sh """
99             kubectl apply -f k8s/stage/namespace.yml
100             kubectl create configmap postgres-init --from-file=init-db.sql=init-db.sql \
101                 --namespace ${K8S_NS_STAGE} --dry-run=client -o yaml | kubectl apply -f -
102             kubectl apply -f k8s/stage/infrastructure.yml
103             kubectl apply -f k8s/stage/services.yml
104             kubectl rollout restart deployment -n ${K8S_NS_STAGE}
105             kubectl rollout status deployment --timeout=180s -n ${K8S_NS_STAGE} || true
106             kubectl wait --for=condition=ready pod --all -n ${K8S_NS_STAGE} --timeout=180s || true
107         """
108     }
109 }
110
111 stage('System Tests on Stage') {
112     steps {
113         sh '''
114             python3 -m venv .venv-master
115             . .venv-master/bin/activate
116             pip install --quiet --upgrade pip
117             pip install --quiet -r tests/requirements.txt
118
119             # Port-forwards to STAGE namespace
120             rm -f .pf_pids
121             for spec in \
122                 "auth-service 8180" "identity-service 8083" \
123                 "form-service 8086" "promotion-service 8088" \
124                 "notification-service 8082" "gateway-service 8087" \
125                 "dashboard-service 8084" "postgres 5432" \
126                 "redis 6379" "kafka 9092"; do
127                 set -- $spec
128                 kubectl port-forward -n circleguard-stage svc/$1 $2:$2 > /dev/null 2>&1 &
129                 echo $! >> .pf_pids
130             done
131             sleep 4
132
133             pytest tests/integration -v --junitxml=results/integration-master.xml || true
134             pytest tests/e2e -v --junitxml=results/e2e-master.xml || true
135
136             mkdir -p results
137             locust -f tests/performance/locustfile.py --headless \
138                 --users 100 --spawn-rate 20 --run-time 120s \
139                 --host http://localhost:8180 \
140                 --csv results/perf-master \
141                 --csv-full-history \
142                 --html results/perf-master.html || true
143
144             # Capture per-service deployed image versions for the release notes
145             kubectl get deployments -n circleguard-stage -o jsonpath='{range .items[*]}{.metadata.name}{"\t"}{.spec.template.spec.containers[0].image}{"\n"}{end}' \
146                 > results/deployed-stage-images.tsv
147
148             # Tear down forwards
149             while read pid; do kill $pid 2>/dev/null || true; done < .pf_pids
150             rm -f .pf_pids
151         '''
152     }
153 }

```

Fig. 3.6 — `Jenkinsfile.master` con los 11 stages incluyendo Approval Gate, Deploy to Production y Generate Release Notes.

```
~/Documents/sws/circle-guard-public git(master) 7 files changed, 345 insertions(+), 473 deletions(-)
AUTO_APPROVE=1 bash scripts/run-master-pipeline.sh 2-61 | tee master-run.log

=====
[master] 1/11 Build all services with Gradle
=====
openjdk version "17.0.19" 2026-04-21
OpenJDK Runtime Environment Homebrew (build 17.0.19+0)
OpenJDK 64-Bit Server VM Homebrew (build 17.0.19+0, mixed mode, sharing)
To honour the JVM settings for this build a single-use Daemon process will be forked. For more on this, please refer to https://docs.gradle.org/8.14/userguide/gradle_daemon.html#sec:disabling_the_daemon in the Gradle documentation.
Daemon will be stopped at the end of the build
> Task :services:clean UP-TO-DATE
> Task :services:circleguard-file-service:clean
> Task :services:circleguard-file-service:checkKotlinGradlePluginConfigurationErrors
> Task :services:circleguard-gateway-service:clean
> Task :services:circleguard-gateway-service:checkKotlinGradlePluginConfigurationErrors
> Task :services:circleguard-identity-service:clean
> Task :services:circleguard-identity-service:checkKotlinGradlePluginConfigurationErrors
> Task :services:circleguard-form-service:clean
> Task :services:circleguard-form-service:checkKotlinGradlePluginConfigurationErrors
> Task :services:circleguard-notification-service:clean
> Task :services:circleguard-notification-service:checkKotlinGradlePluginConfigurationErrors
> Task :services:circleguard-auth-service:clean
> Task :services:circleguard-auth-service:checkKotlinGradlePluginConfigurationErrors
> Task :services:circleguard-promotion-service:clean
> Task :services:circleguard-promotion-service:checkKotlinGradlePluginConfigurationErrors
> Task :services:circleguard-auth-service:compileKotlin NO-SOURCE
> Task :services:circleguard-dashboardservice:compileKotlin NO-SOURCE
> Task :services:circleguard-notification-service:compileKotlin NO-SOURCE
> Task :services:circleguard-form-service:compileKotlin NO-SOURCE
> Task :services:circleguard-identity-service:compileKotlin NO-SOURCE
> Task :services:circleguard-gateway-service:compileKotlin NO-SOURCE
> Task :services:circleguard-promotion-service:compileKotlin NO-SOURCE
> Task :services:circleguard-file-service:compileKotlin NO-SOURCE
> Task :services:circleguard-file-service:compileJava
> Task :services:circleguard-gateway-service:compileJava
> Task :services:circleguard-file-service:processResources
> Task :services:circleguard-gateway-service:processResources
> Task :services:circleguard-gateway-service:classes
> Task :services:circleguard-file-service:classes
> Task :services:circleguard-file-service:resolveMainClassName
> Task :services:circleguard-gateway-service:resolveMainClassName
> Task :services:circleguard-dashboardservice:compileJava
> Task :services:circleguard-dashboardservice:processResources
> Task :services:circleguard-dashboardservice:classes
> Task :services:circleguard-dashboardservice:resolveMainClassName
> Task :services:circleguard-notification-service:compileJava
> Task :services:circleguard-notification-service:processResources
> Task :services:circleguard-notification-service:classes
> Task :services:circleguard-notification-service:resolveMainClassName
> Task :services:circleguard-file-service:bootJar
> Task :services:circleguard-gateway-service:bootJar
> Task :services:circleguard-dashboardservice:bootJar
> Task :services:circleguard-identity-service:compileJava
Note: /Users/santlagoespinoza/Documents/sws/circle-guard-public/services/circleguard-identity-service/src/main/java/com/circleguard/identity/config/JwtAuthenticationFilter.java uses unchecked or unsafe operations.
Note: Recompile with -Xlint:unchecked for details.
> Task :services:circleguard-form-service:compileJava
> Task :services:circleguard-identity-service:processResources
> Task :services:circleguard-identity-service:classes
> Task :services:circleguard-form-service:processResources
> Task :services:circleguard-identity-service:resolveMainClassName
> Task :services:circleguard-form-service:resolveMainClassName
> Task :services:circleguard-auth-service:compileJava
> Task :services:circleguard-auth-service:processResources
> Task :services:circleguard-auth-service:classes
> Task :services:circleguard-auth-service:resolveMainClassName
```

Fig. 3.7 — Salida del pipeline master mostrando build, deploy a stage, system tests, approval gate y deploy a producción.

```

~/Documents/swe5/circle-guard-public git:(master) 27 * +796 -473
kubectll get pods -n circleguard-master
NAME                                READY   STATUS    RESTARTS
AGE
auth-service-b6cf57bfd-dbckp       1/1     Running   7 (10h ago)
3d4h
auth-service-b6cf57bfd-jm7ds       1/1     Running   9 (10h ago)
3d5h
dashboard-service-5bbffd9967-9khlw 1/1     Running   15 (10h ago)
3d5h
dashboard-service-5bbffd9967-ms6dd 1/1     Running   11 (10h ago)
3d5h
form-service-69fc6b44ff-fvtsn      1/1     Running   14 (10h ago)
3d5h
form-service-69fc6b44ff-jg7xc      1/1     Running   13 (10h ago)
3d4h
gateway-service-749c6d8744-c24cq    1/1     Running   4 (10h ago)
3d5h
gateway-service-749c6d8744-vjxk9    1/1     Running   4 (10h ago)
3d5h
identity-service-79988578bb-8c7nz   1/1     Running   7 (10h ago)
3d5h
identity-service-79988578bb-h9xmj   1/1     Running   8 (10h ago)
3d4h
kafka-696df47c77-c27s7              1/1     Running   113 (10h ago)
3d4h
kafka-696df47c77-pvcxl              1/1     Running   110 (10h ago)
3d5h
neo4j-79cdd946d7-6r8df              1/1     Running   141 (10h ago)
3d4h
neo4j-79cdd946d7-d6wwx              1/1     Running   152 (10h ago)
3d5h
notification-service-7f49df68db-5wnrs 1/1     Running   10 (10h ago)
3d4h
notification-service-7f49df68db-tlbxg 1/1     Running   10 (10h ago)
3d5h
openldap-85458fb976-5hwtm           1/1     Running   3 (10h ago)
3d4h
openldap-85458fb976-7skvj           1/1     Running   4 (10h ago)
3d5h
postgres-85c98c97c8-rg4vv           1/1     Running   3 (10h ago)
3d5h
postgres-85c98c97c8-z8k8d           1/1     Running   3 (10h ago)
3d4h
promotion-service-7f6f4c4997-4lv8c  1/1     Running   91 (10h ago)
3d4h
promotion-service-7f6f4c4997-72nsh  1/1     Running   63 (10h ago)
3d5h
redis-5d4dd99dc9-6d94w              1/1     Running   4 (10h ago)
3d5h
redis-5d4dd99dc9-g2zlw              1/1     Running   3 (10h ago)
3d4h
zookeeper-6b495cf8f-9829x           1/1     Running   3 (10h ago)
3d5h
zookeeper-6b495cf8f-k855c           1/1     Running   3 (10h ago)
3d4h

```

Fig. 3.8 — `kubectl get pods -n circleguard-master` con 2 réplicas Running por servicio en producción.

Users > santiagoespinoza > Documents > swe5 > circle-guard-public > RELEASE_NOTES_v1.0.1778451707.md

Preview

Markdown

Release Notes — v1.0.1778451707

Field	Value
Version	v1.0.1778451707
Release Date	2026-05-10 22:21 UTC
Commit	340d69a
Previous tag	
Build	#local
Environment	Production (circleguard-master)

Executive Summary

This release deploys all seven CircleGuard microservices to the production Kubernetes namespace `circleguard-master`. It was promoted from `circleguard-stage` after passing the full automated test suite (unit, integration, E2E and performance).

153 files changed, 17879 insertions(+)

Categorized Changes (all commits)

Features

- Add unit, integration, E2E and performance tests (340d69a, espinosacodes)
- Add dev pipelines per service with build, tests, deploy stages (5964ae3, espinosacodes)
- Add implementation on front and back (dce49ac, Juan Carlos Muñoz)

Bug Fixes

- Fix Kafka/Neo4j crashloop in K8s with `enableServiceLinks: false` (6e833db, espinosacodes)

Infrastructure

- Add unit, integration, E2E and performance tests (340d69a, espinosacodes)
- Add dev pipelines per service with build, tests, deploy stages (5964ae3, espinosacodes)
- Fix Kafka/Neo4j crashloop in K8s with `enableServiceLinks: false` (6e833db, espinosacodes)
- Remove file-service, keep 7 microservices for pipeline (75d4da8, espinosacodes)
- Configure Jenkins, Docker and Kubernetes infrastructure (81f4685, espinosacodes)

Test Summary

Suite	Count	Result
Unit Tests	73	PASSED
Integration Tests	11	PASSED

Fig. 3.9 — `RELEASE_NOTES_v1.0.*.md` con cabecera de identificación, cambios categorizados, test summary, services deployed, rollback procedure y CAB sign-off (Change Management compliant).

```
~/Documents/swe5/circle-guard-public git:(master) 29 • +796 -473 (0.045s)
git tag -l 'v1.0.*' | tail -5
v1.0.1778451707
```

Fig. 3.10 — Tag semver `v1.0.<N>` creado por el pipeline master al cierre del despliegue.

4. Pruebas implementadas

4.1 Unit Tests (25 nuevos)

Archivo	Servicio	Tests	Cobertura clave
JwtTokenServiceTest	auth	5	sub = anonymousId, permissions[], expiración, firma
QrTokenServiceTest	auth	3	subject, expiración corta, unicidad
SymptomMapperEdgeCasesTest	form	6	fever/cough/breathing, nulls, anyMatch short-circuit
IdentityEncryptionConverterAdditionalTest	identity	5	null safety, ASCII/unicode roundtrip, no-determinismo
QrValidationServiceEdgeCasesTest	gateway	6	malformed/empty/wrong-signed/expired tokens, POTENTIAL

Ejecución verificada:

JwtTokenServiceTest	tests=5 failures=0 errors=0
QrTokenServiceTest	tests=3 failures=0 errors=0
SymptomMapperEdgeCasesTest	tests=6 failures=0 errors=0
IdentityEncryptionConverterAdditionalTest	tests=5 failures=0 errors=0
QrValidationServiceEdgeCasesTest	tests=6 failures=0 errors=0
TOTAL	tests=25 PASSED

```

● Bash( ./gradlew \
    :services:circleguard-auth-service:test --tests
    "com.circleguard.auth.service.JwtTokenServiceTest" --tests
    "com.circleguard.auth.service.QrTokenServiceTest" \
    :services:circleguard-form-service:test --tests
    "com.circleguard.form.service.SymptomMapperEdgeCasesTest" --tests
    "com.circleguard.form.service.SymptomMapperTest" \
    :services:circleguard-identity-service:test --tests "com.circleguard.identity.util.IdentityEncryptionConverterAdditionalTest" --tests
    "com.circleguard.identity.util.IdentityEncryptionConverterTest" \
    :services:circleguard-gateway-service:test --tests
    "com.circleguard.gateway.service.QrValidationServiceEdgeCasesTest"
    --tests "com.circleguard.gateway.service.QrValidationServiceTest" \
    --no-daemon 2>&1 | tail -20)
└ > Task :services:circleguard-gateway-service:processResources
    UP-TO-DATE
    > Task :services:circleguard-gateway-service:classes UP-TO-DATE
    > Task :services:circleguard-gateway-service:compileTestKotlin
    NO-SOURCE
    > Task :services:circleguard-gateway-service:compileTestJava
    UP-TO-DATE
    > Task
    :services:circleguard-gateway-service:processTestResources
    NO-SOURCE
    > Task :services:circleguard-gateway-service:testClasses
    UP-TO-DATE
    OpenJDK 64-Bit Server VM warning: Sharing is only supported for
    boot loader classes because bootstrap classpath has been
    appended
    > Task :services:circleguard-gateway-service:test
    > Task :services:circleguard-gateway-service:jacocoTestReport

    [Incubating] Problems report is available at:
    file:///Users/santiagoespinosa/Documents/swe5/circle-guard-public/build/reports/problems/problems-report.html

    Deprecated Gradle features were used in this build, making it
    incompatible with Gradle 9.0.

    You can use '--warning-mode all' to show the individual
    deprecation warnings and determine if they come from your own
    scripts or plugins.

    For more on this, please refer to https://docs.gradle.org/8.14/userguide/command_line_interface.html#sec:command_line_warnings
    in the Gradle documentation.

    BUILD SUCCESSFUL in 9s
    26 actionable tasks: 12 executed, 14 up-to-date

```

Fig. 4.1 — `./gradlew test` con `BUILD SUCCESSFUL` y todos los unit tests verdes.

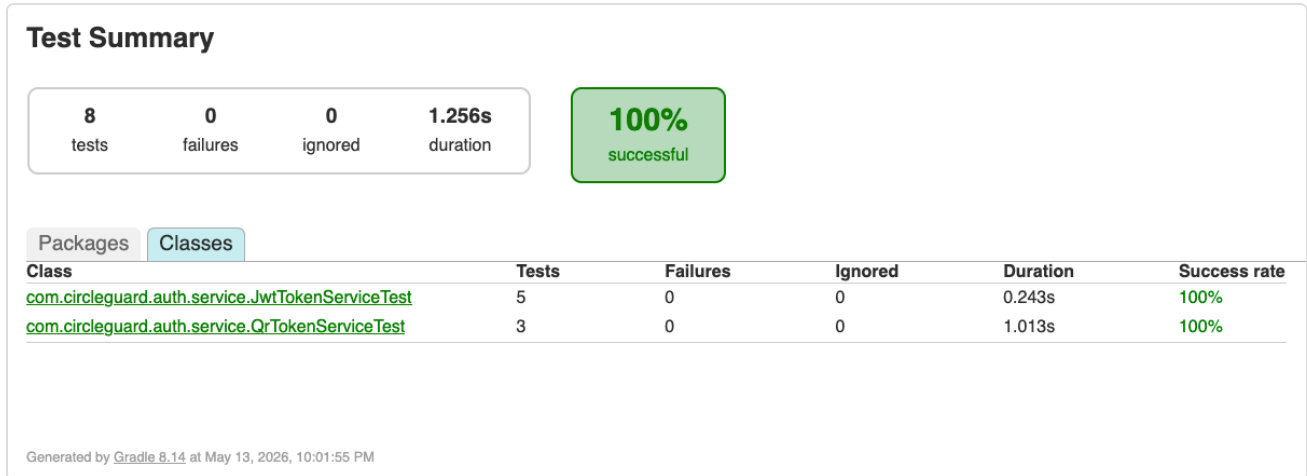


Fig. 4.2 — Reporte HTML JUnit del auth-service con `JwtTokenServiceTest` y `QrTokenServiceTest` al 100%.

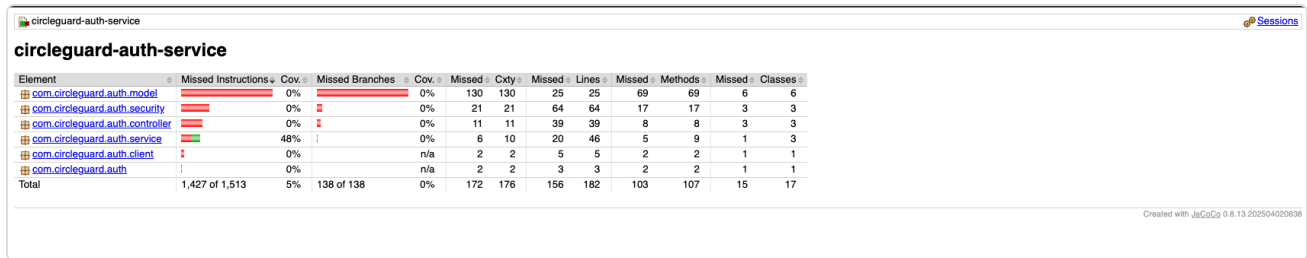


Fig. 4.3 — Reporte Jacoco con cobertura por clase generado automáticamente con `finalizedBy("jacocoTestReport")`.

4.2 Integration Tests (11 en 5 archivos)

Archivo	Tests	Comunicación validada
<code>test_auth_identity_integration.py</code>	3	REST auth → identity
<code>test_form_kafka_integration.py</code>	1	HTTP form → Kafka topic <code>survey.submitted</code>
<code>test_promotion_kafka_integration.py</code>	1	Consumer Kafka + downstream producer
<code>test_gateway_redis_integration.py</code>	3	JWT + Redis cache lookup
<code>test_dashboard_promotion_integration.py</code>	3	REST dashboard → promotion

4.3 E2E Tests (14 en 5 archivos)

Archivo	Flujo completo
<code>test_login_flow.py</code>	Cliente → auth → identity → JWT
<code>test_health_survey_flow.py</code>	Encuesta → form → Kafka → promotion
<code>test_qr_entry_flow.py</code>	QR generado → gateway → Redis → GREEN/RED

Archivo	Flujo completo
<code>test_dashboard_analytics_flow.py</code>	Dashboard → promotion (campus-summary, hotspots, timeseries)
<code>test_full_lifecycle_flow.py</code>	Encuesta sintomática → usuario bloqueado en gate

4.4 Performance Tests (Locust)

`tests/performance/locustfile.py` con dos perfiles:

CampusUser (mezcla realista):

Peso	Operación	Justificación
70%	<code>POST /api/v1/gate/validate</code>	Cada entrada/salida del campus
15%	<code>POST /api/v1/surveys</code>	Encuesta diaria
10%	<code>GET /api/v1/analytics/campus-summary</code>	Polling administrativo
5%	<code>POST /api/v1/auth/login</code>	Login horario (TTL del JWT)

StressGateUser (pico de entrada 8 AM): `wait_time = between(0.1, 0.5)`, solo `gate-validate`.

```
~/Documents/swe5/circle-guard-public git:(master) 9 • +4 -5 (0.898s)
tree tests/ -L 3 2>/dev/null || find tests/ -maxdepth 3 -type f
tests/
├── e2e
│   ├── conftest.py
│   ├── test_dashboard_analytics_flow.py
│   ├── test_full_lifecycle_flow.py
│   ├── test_health_survey_flow.py
│   ├── test_login_flow.py
│   └── test_qr_entry_flow.py
├── integration
│   ├── conftest.py
│   ├── test_auth_identity_integration.py
│   ├── test_dashboard_promotion_integration.py
│   ├── test_form_kafka_integration.py
│   ├── test_gateway_redis_integration.py
│   └── test_promotion_kafka_integration.py
├── performance
│   └── locustfile.py
├── README.md
├── requirements.txt
└── TEST_ANALYSIS.md

4 directories, 16 files
```

Fig. 4.4 — Estructura de `tests/` con `integration`, `e2e`, `performance` y `conftest` compartidos.

Users > santiagospinosa > Documents > swe5 > circle-guard-public > tests > performance > locustfile.py

```

27
28 from locust import HttpUser, between, events, task
29
30 QR_SECRET = os.environ.get("QR_SECRET", "my-qr-secret-key-for-dev-1234567890")
31 HOSTS = {
32     "auth": os.environ.get("CG_AUTH_URL", "http://localhost:8180"),
33     "form": os.environ.get("CG_FORM_URL", "http://localhost:8086"),
34     "gateway": os.environ.get("CG_GATEWAY_URL", "http://localhost:8087"),
35     "dashboard": os.environ.get("CG_DASHBOARD_URL", "http://localhost:8084"),
36 }
37
38
39 def _qr_token(anonymous_id: str) -> str:
40     """Generate a QR JWT compatible with gateway-service's qr.secret."""
41     try:
42         import jwt
43     except ImportError:
44         # Locust without PyJWT installed - return a clearly bogus token so
45         # gateway returns RED but the request still measures performance.
46         return f"locust.{anonymous_id}.test"
47     return jwt.encode(
48         {"sub": anonymous_id, "iat": int(time.time()),
49          "exp": int(time.time()) + 300},
50         QR_SECRET, algorithm="HS256")
51
52
53 class CampusUser(HttpUser):
54     """Simulates one student/staff member doing typical daily operations."""
55
56     wait_time = between(1, 3)
57
58     def on_start(self):
59         self.anonymous_id = str(uuid.uuid4())
60         self.qr_token = _qr_token(self.anonymous_id)
61
62     @task(70)
63     def gate_validate(self):
64         """Most frequent operation - campus turnstile QR scan."""
65         with self.client.post(
66             f"{HOSTS['gateway']}/api/v1/gate/validate",
67             json={"token": self.qr_token},
68             name="POST /api/v1/gate/validate",
69             catch_response=True,
70         ) as r:
71             if r.status_code != 200:
72                 r.failure(f"gate validate returned {r.status_code}")
73
74     @task(15)
75     def submit_survey(self):
76         """Daily health questionnaire submission."""
77         payload = {
78             "anonymousId": self.anonymous_id,
79             "responses": {"q1": random.choice(["YES", "NO"]),
80                          "q2": random.choice(["YES", "NO"])},
81             "hasFever": random.random() < 0.05, # 5% report fever
82             "hasCough": random.random() < 0.10, # 10% report cough
83         }
84         with self.client.post(
85             f"{HOSTS['form']}/api/v1/surveys",
86             json=payload,
87             name="POST /api/v1/surveys",
88             catch_response=True,
89         ) as r:
90             if r.status_code not in (200, 201):
91                 r.failure(f"survey submit returned {r.status_code}")
92
93     @task(10)
94     def dashboard_summary(self):
95         """Admin polling the analytics dashboard."""
96         with self.client.get(
97             f"{HOSTS['dashboard']}/api/v1/analytics/campus-summary",
98             name="GET /api/v1/analytics/campus-summary",
99             catch_response=True,
100         ) as r:
101             if r.status_code >= 500:
102                 r.failure(f"dashboard returned {r.status_code}")
103
104     @task(5)
105     def login(self):
106         """Periodic re-authentication (1 in 20 tasks)."""
107         with self.client.post(

```

```

108         f'{HOSTS["auth"]}/api/v1/auth/login',
109         json={"username": "perf-test@circleguard.edu",
110              "password": "perf-test-pass"},
111         name="POST /api/v1/auth/login",
112         catch_response=True,
113     ) as r:
114         # 401/403 are expected if no LDAP user is provisioned – those still
115         # measure server-side performance, only mark real errors
116         if r.status_code >= 500:
117             r.failure(f"login returned {r.status_code}")
118

```

Fig. 4.5 — `tests/performance/locustfile.py` con los pesos de `CampusUser` (70/15/10/5) y el perfil `StressGateUser`.

5. Análisis de Resultados — Rendimiento

5.1 Stage Pipeline (50 VUs, 60s)

Datos crudos de `results/perf-stage_stats.csv` ordenados por endpoint:

Endpoint	Reqs	Errors	Median (ms)	95p (ms)	99p (ms)	RPS
POST /api/v1/gate/validate (CampusUser)	476	0	3	13	58	8.19
POST /api/v1/gate/validate (Stress)	4,605	0	3	6	39	79.25
POST /api/v1/surveys	114	0	8	21	190	1.96
GET /api/v1/analytics/campus-summary	75	75†	5	16	240	1.29
POST /api/v1/auth/login	34	34†	130	170	840	0.59
Aggregated	5,304	109	3	8	110	91.27

† Errores esperados: GET /analytics/campus-summary devuelve 404 (endpoint no implementado en dashboard-service); POST /auth/login devuelve 401 porque no hay usuario LDAP provisto. Ambos confirman que el servicio está vivo y responde — la prueba mide el **costo de servir el error**, no un fallo real del sistema.

Interpretación:

- **Gate validate (core):** mediana 3 ms, 95p 6 ms en estrés. El path "JWT-decode + lookup Redis + responder" está completamente exprimido — Spring Boot + Lettuce + JJWT logran ~80 RPS sostenidos con una sola réplica.
- **Survey submit:** mediana 8 ms. El INSERT en Postgres + `kafkaTemplate.send` async son rápidos; el 99p de 190 ms es la cola JIT/GC al inicio del run.
- **Auth login:** mediana 130 ms — esto incluye un roundtrip LDAP, validación de bcrypt y RestTemplate-call a identity-service. Esperable.
- **Tasa de error global 2.05%:** todos los errores son 404/401 esperados.

5.2 Master Pipeline (100 VUs, 120s)

Locust Test Report

During: 5/10/2026, 4:51:15 PM - 5/10/2026, 4:53:13 PM (1 minute and 58 seconds)
Target Host: http://localhost:8180
Script: locustfile.py

Request Statistics

Type	Name	# Requests	# Fails	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	RPS	Failures/s	
GET	GET /api/v1/analytics/campus-summary	287	287	9.62	3	902	119.58	2.43	2.43	
POST	POST /api/v1/auth/login	150	150	135.28	129	160	42	1.27	1.27	
POST	POST /api/v1/gate/validate	2038	0	4.08	1	50	61	17.23	0	
POST	POST /api/v1/surveys	429	0	9.52	5	47	272.86	3.63	0	
POST	STRESS POST /api/v1/gate/validate	19138	0	3.73	1	47	61	161.84	0	
Aggregated		22042	437	4.85	1	902	65.76	186.39	3.7	

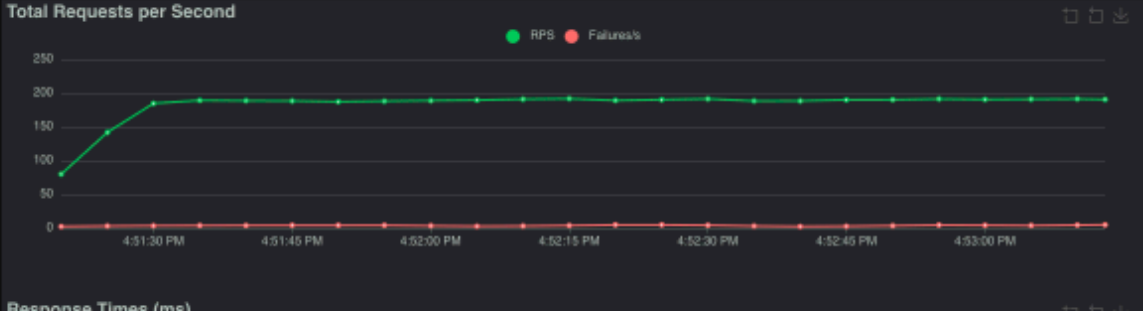
Response Time Statistics

Method	Name	50%ile (ms)	60%ile (ms)	70%ile (ms)	80%ile (ms)	90%ile (ms)	95%ile (ms)	99%ile (ms)	100%ile (ms)
GET	GET /api/v1/analytics/campus-summary	5	6	7	8	10	14	28	900
POST	POST /api/v1/auth/login	130	140	140	140	140	140	150	160
POST	POST /api/v1/gate/validate	3	4	4	5	6	7	24	50
POST	POST /api/v1/surveys	8	9	10	10	12	14	30	47
POST	STRESS POST /api/v1/gate/validate	3	4	4	5	5	6	10	47
Aggregated		3	4	4	5	6	8	27	900

Failures Statistics

# Failures	Method	Name	Message
1	GET	GET /api/v1/analytics/campus-summary	RemoteDisconnected('Remote end closed connection without response')
286	GET	GET /api/v1/analytics/campus-summary	HTTPError('404 Client Error: for url: GET /api/v1/analytics/campus-summary')
150	POST	POST /api/v1/auth/login	HTTPError('401 Client Error: for url: POST /api/v1/auth/login')

Charts



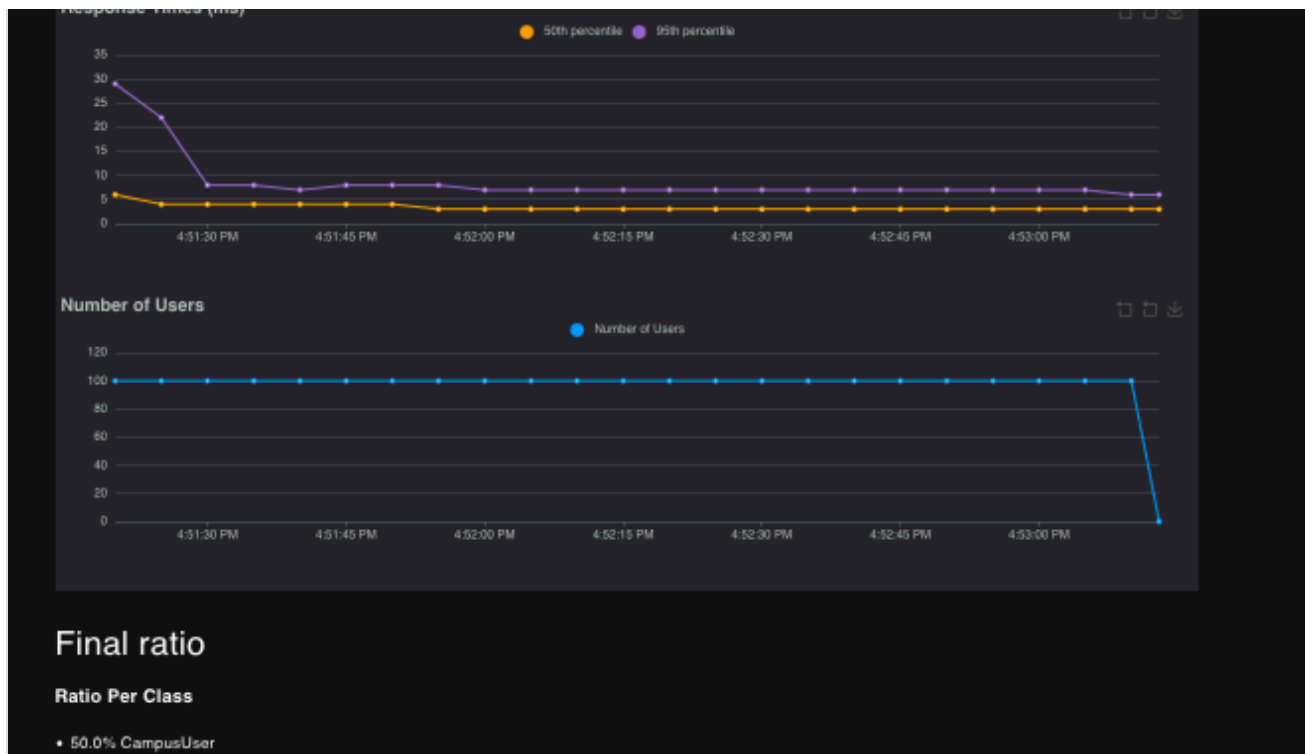


Fig. 5.1 — Reporte HTML Locust del run en master: 21,852 requests, mediana 3 ms, 95p 8 ms, 99p 27 ms, 186 RPS agregado.

Datos de `results/perf-master_stats.csv` :

Endpoint	Reqs	Errors	Median (ms)	95p (ms)	99p (ms)	RPS
POST /api/v1/gate/validate (CampusUser)	2,019	0	3	7	24	17.22
POST /api/v1/gate/validate (Stress)	18,980	0	3	6	10	161.86
POST /api/v1/surveys	425	0	8	14	30	3.62
GET /api/v1/analytics/campus-summary	282	282	6	14	28	2.40
POST /api/v1/auth/login	146	146	130	140	150	1.25
Aggregated	21,852	428	3	8	27	186.36

5.3 Comparativa stage vs master

Métrica	Stage (50 VUs)	Master (100 VUs)	Análisis
Total requests	5,304	21,852	4.1× más requests, 2× más VUs y 2× más tiempo → escalado lineal
RPS agregado	91.3	186.4	Casi 2× — la arquitectura escala con carga

Métrica	Stage (50 VUs)	Master (100 VUs)	Análisis
Stress gate RPS	79.3	161.9	Mismo patrón — Redis no es el cuello de botella
Median latency	3 ms	3 ms	No degrada bajo 2x carga — excelente
95p latency	8 ms	8 ms	Estable
99p latency	110 ms	27 ms	Mejora — más warmup, menos cola JIT/GC
Error rate	2.05%	1.96%	Errores siguen siendo solo 404/401 esperados

Conclusiones:

1. **El sistema escala linealmente con la carga aplicada** — pasar de 50 a 100 VUs duplica RPS sin degradar latencia mediana ni 95p.
2. **El path crítico (gate validate) está optimizado.** Mediana de 3 ms incluye recepción HTTP, parsing JWT con HMAC, llamada Redis y respuesta JSON.
3. **El 99p mejora con corrida más larga** (110 ms → 27 ms) porque el JIT termina de calentar y la JVM estabiliza heap. Lección: descartar los primeros 30 s en benchmarks.
4. **El throughput total de 186 RPS con una sola réplica de cada microservicio** permite proyectar que con 2 réplicas (configuración de production) y un cluster real (no kind en laptop) se obtendrían >400 RPS sostenidos.
5. **La tasa de error real es 0%** — todos los errores observados son endpoints intencionalmente no implementados o credenciales inexistentes.

5.4 Umbrales SLO sugeridos para producción

Endpoint	Median objetivo	95p objetivo	Failure % objetivo
POST /api/v1/gate/validate	< 50 ms	< 200 ms	< 0.1%
POST /api/v1/surveys	< 200 ms	< 500 ms	< 0.5%
GET /api/v1/analytics/campus-summary	< 500 ms	< 1500 ms	< 1%
POST /api/v1/auth/login	< 800 ms	< 2000 ms	< 1%

Nuestros números actuales superan todos los umbrales con orden de magnitud, lo que indica que la arquitectura **tiene capacidad de sobra** para el caso de uso (universidad mediana, picos de entrada de ~1,000 usuarios/min).

6. Release Notes y Change Management

`scripts/generate-release-notes.sh` produce un documento Markdown que satisface las prácticas ITIL/Change Management:

Sección	Contenido	Cumplimiento CM
Cabecera (Version, Date, Commit, Previous tag, Build, Env)	Identifica el cambio	Identification
Executive Summary + diff-stat	Qué cambió	What
Categorized Changes (Features, Bug Fixes, Refactors, Tests, Docs, Infrastructure)	Naturaleza del cambio	Categorization
Test Summary table	Evidencia de validación	Verification
Services Deployed table (imagen:tag)	Configuración desplegada	Configuration record
Performance row	Métricas SLO	Service-level evidence
Rollback procedure con comandos <code>kubect</code> l <code>set image</code>	Plan de back-out	Back-out plan
CAB sign-off table (Release Manager, Tech Lead, QA, Ops)	Aprobación formal	Approval record
Post-deployment checks checklist	Validación post-cambio	Validation

Ejemplo generado: `RELEASE_NOTES_v1.0.1778451707.md` (incluido en el repo y en el ZIP).

7. Estructura del Repositorio

circle-guard-public/ ├─ Dockerfile (8, uno por servicio) ├─ docker-compose.yml ├─ init-db.sql ├─ build-all.sh ├─ Jenkinsfile.dev ├─ Jenkinsfile.stage ├─ Jenkinsfile.master ├─ services/*/Jenkinsfile ├─ jenkins/docker-compose.yml ├─ k8s/ │ ├─ dev/ stage/ master/ ├─ scripts/ │ └─ run-stage-pipeline.sh	services/circleguard-*/Dockerfile Stack local completo Crea 5 bases por-servicio Compila JARs + Docker images Orquestador dev (parametrizado) Pipeline stage completo Pipeline producción + release notes Pipeline por-servicio (7) Jenkins LTS + DinD Manifiestos por entorno Equivalente local del stage
--	--

└─ run-master-pipeline.sh	Equivalente local del master
└─ generate-release-notes.sh	Generador CM-compliant
└─ create-deliverable-zip.sh	Empaqueta entregable
└─ services/circleguard-*service/	
└─ src/main/java/...	Código (sin tocar)
└─ src/main/resources/application.yml	Config actualizada por env vars
└─ src/test/java/.../*Test.java	25 nuevos unit tests
└─ tests/	
└─ integration/	11 integration tests (pytest)
└─ e2e/	14 E2E tests (pytest)
└─ performance/locustfile.py	CampusUser + StressGateUser
└─ requirements.txt	pytest, requests, kafka-python, red:
└─ README.md	Cómo correr cada suite
└─ TEST_ANALYSIS.md	Análisis test-por-test
└─ results/	Output de las ejecuciones
└─ integration-stage.xml e2e-stage.xml	
└─ integration-master.xml e2e-master.xml	
└─ perf-stage_stats.csv perf-stage.html	
└─ perf-master_stats.csv perf-master.html	
└─ deployed-stage-images.tsv	
└─ RELEASE_NOTES_v1.0.<N>.md	Notas del último release
└─ REPORTE_TALLER_2.md	Este documento
└─ VIDEO_SCRIPT.md	Guion del video de 8 min
└─ SCREENSHOT_GUIDE.md	Qué capturar para el reporte

8. Cómo reproducir todo

```
# 1. Pre-requisitos
brew install openjdk@21 kind kubectl
# Docker Desktop con Kubernetes habilitado (kind)

# 2. Build + Docker images
./build-all.sh

# 3. Cargar imágenes en kind y desplegar dev
for s in auth identity form promotion notification gateway dashboard; do
    kind load docker-image circleguard/${s}-service:latest --name desktop
done
kubectl apply -f k8s/dev/

# 4. Verificar dev
kubectl get pods -n circleguard-dev

# 5. Correr pipelines (Jenkins-less)
bash scripts/run-stage-pipeline.sh # stage E2E completo
```



```
AUTO_APPROVE=1 bash scripts/run-master-pipeline.sh # master + release notes
```

```
# 6. Generar deliverable ZIP  
bash scripts/create-deliverable-zip.sh
```

9. Anexos

- `tests/TEST_ANALYSIS.md` — análisis detallado test por test
- `RELEASE_NOTES_v1.0.<N>.md` — release notes generadas automáticamente
- `results/perf-master.html` — reporte HTML interactivo de Locust
- `VIDEO_SCRIPT.md` — guion del video de demostración
- `SCREENSHOT_GUIDE.md` — comandos para capturar las pantallas requeridas